

DynamiX Device Reference

Checked against the live device registry

September 28, 2026 · kernel figures drawn by `make_figures.py` from the app’s own kernel code

How to read this document

A *device* is one processing tool in a layer’s chain. **Transforms** compute (they head chains and produce a result bundle); **filters** select from a computed result (list indexes and masks — scrubbing a filter knob redraws without recomputing). Chains are always transforms-then-filters; interleaving is rejected at construction.

Conventions that hold everywhere:

- **Scales are stored in pixels**, converted only for display.
- **The wavelet family belongs to the method**: each tool’s menu lists only the kernel class its method admits, under the method’s own names.
- **Inert knobs**: a parameter that binds only under one choice of another parameter (e.g. q only for q-family wavelets) is ignored otherwise; where the device declares it, the knob is *greyed out* in the device box while it does not apply (its value is kept).
- **View-only knobs** (marked *view* in the tables) choose what is shown from a result that is already computed and cached — Show, Component, Group, Orientation, Show PC. Switching one redraws at once; it never recomputes.
- **Orientations are (angle, frame)**, the frame being the pixel grid or grid, true or magnetic north (§4.1, Gradient arrows and anisotropy).
- **No-support masking**: pixels with no honest signal support (nodata flats, FFT dust) are NaN in every exponent map.
- Each table lists: parameter (UI label), default, hard range, and meaning. Soft ranges (knob-drag limits) are narrower than hard ranges.

Wavelet normalization: L^1

Every wavelet in the app is normalized in the L^1 (pure-dilation) convention, $\psi_a(x) = a^{-d} \psi(x/a)$ in $d = 2$ dimensions, so that at a Hölder singularity the transform scales as $|T_\psi[f](x, a)| \sim a^{h(x)}$ with no offset. The L^2 convention ($a^{-d/2}$, unit energy $\int \psi^2 = 1$ — e.g. the normalization constants A and A_q of Borges et al. 2004, whose transform carries $a^{-1/2}$) shifts every exponent by $d/2$. Turiel’s microcanonical formalism uses the same L^1 dilation $\Psi_r(x) = r^{-d} \Psi(x/r)$ (Turiel et al. 2008, eqs. 6–8). Borges et al.’s own WTMM example agrees in practice: their eq. 41 reads $T \sim a^\alpha$ (their fig. 5 recovers $\alpha = 0.5$), which holds for L^1 scaling, not for the $a^{-1/2}$ of their eq. 10.

- **wtmm2d normalizes in the Fourier domain**. Each filter is built as $\hat{\psi}(a\mathbf{k})$ from a smoothing function with $\hat{\theta}(0) = 1$ (unit mass at every scale), times the derivative factor $i a\mathbf{k}$: scale enters only through $a\mathbf{k}$, which is the L^1 dilation.
- **The Hölder tools normalize in real space** by default, on the pixel grid: positive kernels (`holder_measure`) to unit discrete mass; Mexican hats (`holder_multiaffine`) to exact zero mean, then unit L^1 norm

($\sum |\psi_r| = 1$), at every scale. With *Wavelet space* set to Fourier (§6) they are built as $\hat{\psi}(r\mathbf{k})$ instead, like `wtmm2d`'s filters.

- A constant prefactor that does not depend on the scale (such as A or A_q) cannot change any exponent — it only shifts $\log |T|$ — and the app's own normalization replaces it anyway. Only the *scale dependence* of the normalization matters.
- **Validated on a synthetic cusp** $|x - x_0|^h$: both routes recover the slope h (tests pin it; an L^2 slip would read $h + 1$ in 2-D). The smallest scales are biased by the discretization of both kernel and signal: at a sampled cusp the slope settles above $\sigma \approx 5$ px (`wtmm2d`) and $r \approx 10$ px (the Mexican hats), later for sharp-cutoff kernels ($q_T < 1$).
- **Two different q 's.** q_T (the knob *Tsallis* q of the Hölder tools) is the Tsallis/Borges shape parameter of a kernel; q in $Z(q, a)$, $\tau(q)$ and $D(h)$ is the moment order of the partition functions. They are unrelated.

Contents

1	The workspace: datasets, layers, ROIs, derivatives	3
2	Field preparation	6
2.1	noise — dither / perturbation	6
2.2	band_select — one band of a stack	6
3	Routing	6
3.1	bus — sends into one stack	6
4	Primary analyzers	6
4.1	wtmm2d — canonical WTMM (Arnéodo)	6
4.2	wtmm2d_roi — WTMM over one ROI window	7
4.3	mz_edges — Mallat–Zhong dyadic multiscale edges	7
4.4	wavelet_skeleton — medial-axis (modulus-minima) extractor	9
4.5	cdf_edges — complex cross-diffusion edges	9
4.6	pm_edges — Perona–Malik anisotropic diffusion edges	10
5	Decompositions	10
5.1	pca — principal components of a multi-band field	11
5.2	tucker_H00I_H0SVD — delay-embedded Tucker decomposition (Tucker-SSA)	11
5.3	ssa2d — 2D singular-spectrum analysis	12
6	Microcanonical tools	13
6.1	holder_measure — gradient-measure route	13
6.2	holder_multiaffine — wavelet-projection route	14
6.3	band_recon_measure / band_recon_multiaffine — reconstruction from an h band	16
7	Topology	17
7.1	chain_topology (Transform)	17
7.2	min_vchains (Filter)	17
8	Extrema filters	17
9	Chain filters (WTMM maxima lines)	18
9.1	chain_classify	18
10	Groups, points, dev	18
11	Planned: q-generalized wavelets	18

1 The workspace: datasets, layers, ROIs, derivatives

Datasets. Every opened file is a *dataset*: one row in the layer tree, and that row *is* its raw layer (its H hides the raster; right-click → Remove dataset takes the dataset and everything on it). A plain image (PNG/JPEG) opens as one luminance band; a multiband GeoTIFF / netCDF / HDF stack opens as (n_y, n_x, n_c) .

Children. A tool dropped on a dataset never replaces the data: it spawns a *child* layer holding that tool, nested under the dataset. A child re-runs its chain on its parent’s data whenever that changes. Right-click on a result row: *Delete layer* (its children move up to its parent) or *Delete layer and children*; shift-click or Cmd-click (Ctrl on Windows) selects several rows for one Delete. **H never cascades:** each row hides only its own drawing (the dataset’s H hides the dataset raster, not the results over it).

ROIs. A saved region of interest is stored in whole *file* pixels and shown as a row **dataset @A** under its dataset. Selecting the ROI row makes it the region the next tool runs on; results (**tool @A**) nest under it. The ROI row’s H hides only its outline. An ROI cannot be deleted while results use it (delete them first). Analyzers read their own declared margin around the ROI; decompositions (**pca**, **tucker_H00I_H0SVD**, **ssa2d**) analyse the ROI itself (margin 0: a margin would mix outside data into the statistic).

Output rows. A result whose tool produces several things (**cdf_edges**, **pm_edges**, **mz_edges**) carries a collapsed *Outputs* group under its row: one row per output, each with its own H. Raster outputs share the canvas one at a time: showing one hides the others, and hiding the one shown puts the data back. They follow the tool’s **show** knob, and switching between them re-uses what was computed. The *edges* row hides the maxima dots and gradient arrows (and the H-lines in the Vector and Geo views); the result keeps its maxima, so filters, the spectrum and the anisotropy window still read them. The layer’s own H hides everything the result draws. An output that needs its own computation (the **mz_edges** reconstruction) runs the first time it is shown, with progress and Stop, and stays cached; Freeze (the row’s F) keeps it.

Progress is live: every compute reports one increasing fraction from inside its own loops.

The right panel shows what applies to the selected row: the colours, widths and arrows of the WTMM overlays appear with a result that has maxima or chains, *Reconstruct...* with an *h* map, colormap, stretch and hillshade with a single-band display (a multiband display uses the Composite section), and the view-specific sections in the Vector and Geo views.

Derivative datasets (Fork)

A *child* keeps re-processing its parent. A **derivative** is forked once from what a result shows and from then on is an *original raw dataset*: nothing links it back, and a tool dropped on it spawns a child that analyses the derivative’s own pixels.

Where	result row → right-click → <i>Fork derivative dataset...</i> ; or the <i>Fork derivative...</i> button of the Components window
Raster	a checklist of every raster the result holds: the one on screen first (after its view knobs — e.g. Group “1”, Show residual = data – C1), then the components / PCs, recon and residual per input band of a multiband field, filtered, $h(x)$...; every checked raster becomes a band (n checked → an n -band dataset)
Vector	the extrema and maxima lines the result shows (after its filters)
Both	raster and vector go in one file
Placement	a dataset row of its own, or nested inside the dataset it came from (removing that dataset lifts the derivative to the top)

Keep	<i>Save to a file...</i> (a Save-As dialog; permanent), or <i>Temporary</i> : a file in this session's scratch folder, left out of a saved project (a notice names it) until <i>Save derivative as...</i> on its row writes it where you choose
Grid	the result's own grid, never resampled: the whole field, or the ROI window for an ROI result
Georeference	the parent's CRS rides over (with the grid's axes); the parent file's layout (its source path, window, full size) does not
File	a RasterField .npz (values, axes, frame, units, provenance) — it opens like any .npz ; vectors ride under vec.* keys
History	provenance["derived"] : parent path and SHA-256, the layer, its whole chain with every parameter (view knobs included), the ROI window, the date; provenance["bands"] names the bands

A derivative that carries vectors gets **derived_vectors** (a loader the shell places, in the browser's Dev group) as its first step: its frozen extrema and maxima lines draw like any result's, and the extrema and chain filters can be dropped after it.

The Components window

Right panel → Decomposition → *Components...*, on a **ssa2d** or **tucker_HOOI_HOSVD** result:

- **thumbnails** — the eigenarrays (**ssa2d**: the $L_r \times L_c$ window patterns) or the component images, each labelled with its share;
- **the shares** on a log scale (group members in amber) — a slowly decaying floor is noise;
- **the w-correlation matrix** (**ssa2d**; black 0 to white 1) — separable components are nearly w-orthogonal, so groups show as blocks on the diagonal; hovering reads $w(C_i, C_j)$.

Clicking thumbnails picks the **group**: click one, Cmd-click (Ctrl) to add or remove, shift-click for a range, *All* for every one. The pick is written into the tool's Group knob (and the window follows the knob), so the canvas redraws from the cache.

Multidimensional rasters

Import. Opening a multi-grid container (netCDF, HDF5, HDF4) shows an import dialog: the image grids grouped by sensor and resolution with a checkbox per band, and the remaining grids (geolocation lattices, calibration tables, supplements) in a collapsed *Ancillary grids* group, each importable on its own. Each ticked group becomes one multiband dataset (n_y, n_x, n_c) . Grids of different resolution are separate datasets: ASTER's VNIR (15 m), SWIR (30 m) and TIR (90 m) arrive as three. HDF4 grids are named by their swath group (VNIR.Band3N/ImageData), since ASTER names every band's array **ImageData**. A dataset is its file together with its band list, so two groups imported from one file are two datasets, each with its own row, cache and saved state.

Band rows. A multiband dataset lists its bands under a collapsed *Bands (n)* row. Each band row is a child layer whose chain starts with **band_select**: selecting it shows that band, and a tool dropped on it runs on that band alone. *Remove band from project* (right-click, or Delete) takes the band out of the dataset together with its row and any results under it; the file is untouched, and the last band stays. The removal changes the dataset's identity, so a result computed on the fuller stack is recomputed.

Composite. A multiband dataset draws as an RGB composite. The *Composite* section of the right panel lists the bands with a channel (R, G, B or none; each channel takes one band) and Solo / Mute buttons: mute drops a band’s channel; one solo shows that band alone in grey; several solos keep each in its channel. Its *Stretch* applies one type to all channels, each computed from its own band’s values:

Stretch	Mapping to $[0, 1]$
linear (min–max)	minimum to maximum (normalisation)
percent clip	the p -th to $(100 - p)$ -th percentiles, clipped ($p = 2$ is ENVI’s linear 2 %)
std-dev	mean $\pm k\sigma$, clipped
log	$\log(1 + v - \min v)$, then minimum to maximum
histogram equalize	each value’s rank among the finite values
bipolar	$-m$ to $+m$, m the $(100 - p)$ -th percentile of $ v $: zero maps to mid-scale, for signed data (principal components, residuals)

The single-band Display stretch offers the same six types. Stretches are display only; analysis reads the band values themselves.

What a field stage shows. A chain that ends on a field stage shows the field it produces: **noise** alone shows the noised field (a stack through the composite), a band row its band, a bus its stack. Bypassing the stage shows the raw data again, and *Fork derivative dataset...* takes the produced field, one band per checkbox.

Buses: routing bands into one stack

Multi-band tools (**pca**, **tucker_HO0I_HOSVD**) take a stack (n_y, n_x, n) . A **bus** assembles one from the bands of any datasets on the same grid. It heads a child layer’s rack (**bus**), and a tool dropped after it analyses the bus’s stack. Right-click a dataset row $\rightarrow$ *Route bands to a new bus...*:

- **Sends** are picked per band from every loaded dataset; a dataset on another grid is listed greyed out with the reason. A send references the band in its file (the file path and the band’s grid id or index); the bus reads the band when it computes, over the window it needs.
- **Channel order** (drag to reorder) is the band axis the tool receives. **pca** and **tucker_HO0I_HOSVD** give the same decomposition in any order, with the per-band loadings in that order; band math and the composite’s default channels depend on it.
- **Same grid**: every send has the bus’s shape, pixel step, georeference anchor and CRS. Combining grids of different resolution requires resampling, which the bus does not do.
- **Windows**: on an ROI the bus reads each band over the ROI’s processing window, reflect-padded at the file edges exactly as the dataset itself is.
- **Returns**: the results of the tool after the bus are ordinary result rows; forking one makes it a dataset of its own.
- *Edit bus sends...* (right-click the bus row) re-opens the routing; a routing already computed is answered from the cache.

Live band stacks. Select two or more layers (shift- or Cmd-click), right-click $\rightarrow$ *Build band stack from N selected layers*: a bus whose sends are the rasters those layers show (a reconstruction, a noised band, an h map), referenced live. Each send carries a fingerprint of its layer’s chain; editing a referenced layer changes the fingerprint, and the stack recomputes when next shown. Referenced layers are computed first when needed, and a deleted one is reported by name. The layers share the stack’s grid (an ROI result has its own grid and is refused). Layer sends and file sends mix in one bus.

2 Field preparation

2.1 noise — dither / perturbation

Adds controlled noise to the field before analysis. Its main use is *dithering* quantized data ($\pm \frac{1}{2}$ LSB) so that quantization plateaus do not read as zero-gradient flats.

Parameter	Default	Range	Meaning
dist (Distribution)	uniform	uniform / gaussian	noise law
amplitude	0	[0, 10 ⁹]	0 = auto $\frac{1}{2}$ LSB; uniform: half-width; gaussian: σ
seed	0	[0, 2 ³¹]	RNG seed (deterministic reruns)
sign (Apply)	add	add / subtract	subtract undoes a previous add exactly

2.2 band.select — one band of a stack

Slices one band out of a multiband field as the working field; grid, frame and CRS are unchanged. Band rows start with it and name their band, so removing another band from the dataset leaves each row on its own band.

Parameter	Default	Range	Meaning
band (Band)	1	[1, 256]	the band, 1-based (a band row selects by name)

3 Routing

3.1 bus — sends into one stack

The head of a bus layer’s rack (§1, Buses). It has no knobs: the routing dialog writes its send list. Its output is the stack (n_y, n_x, n) in channel order, or one band for one send, on the incoming field’s grid (the whole dataset, or an ROI’s processing window). The shell places it; it is not in the device browser.

4 Primary analyzers

Each primary analyzer is its own representation of the field. Dropping a *different* primary analyzer onto an analyzed layer forks a sibling layer rather than mutating the chain; consumers (filters) follow their producer on forks.

4.1 wtmm2d — canonical WTMM (Arnéodo)

The canonical multifractal machinery: a continuous wavelet transform over a geometric scale grid, modulus *maxima* detected per scale (along-gradient non-maxima suppression, or xsmurf’s **follow**), maxima chained across scales into maxima lines, feeding partition functions $\rightarrow \tau(q) \rightarrow$ Legendre $D(h)$. The maxima are dense by design — they are the statistical support, not a lineament map (use the filters, or the other analyzers, for sparse features). Legendre spectra are concave hulls by construction; the microcanonical tools (§6) measure $D(h)$ directly — together they diagnose non-concavity.

Parameter	Default	Range	Meaning
n_oct (Octaves)	3	[1, 12]	scale range: a spans n_oct octaves above $a_{\min}$
n_voice	4	[1, 64]	scales per octave (grid density)
a_min ($a_{\min}$)	1.0	[0.01, 64] px	finest analyzed scale
wavelet	mexican	mexican / gaussian	CWT kernel (Mexican hat = LoG; gaussian = gradient pair)

<code>dither</code>	off	bool	$\pm \frac{1}{2}$ LSB dither for quantized rasters
<code>fracint_alpha</code> (η)	1.0	$[-6, 6]$	per-scale a'' lift (pseudo fractional integration; shifts effective h by η)
<code>detector</code> (Detector)	nms	nms / follow	maxima detection: non-maxima suppression along the gradient, or xsmurf's follow (contour following of the zero-crossings of κ)
<code>interpolate</code>	off	bool	subpixel refinement of each maximum along its gradient (follow always refines)
<code>min_chain_len</code>	2	$[1, 1024]$	discard maxima lines spanning fewer scales
<code>smooth</code>	on	bool	smooth maxima-line traces
<code>thresh</code>	0.001	$[0, 1]$	modulus floor for maxima detection (fraction)
<code>dist2_max</code>	50	$[0, 10^6]$ px ²	max squared jump when chaining maxima across scales
<code>box_ratio</code>	1.0	$[0.001, 64]$	chaining search-box anisotropy
<code>similitude</code>	0.8	$[0, 1]$	cross-scale similarity acceptance for chaining

Gradient arrows and anisotropy

The argument $\mathcal{A} = \arg T_\psi$ of each maximum points uphill in the pixel frame: 0° toward increasing column, $+90^\circ$ toward increasing row (clockwise on screen). **Gradient arrows** (Display section: off / on WTMM / on all maxima) draw one arrow per point at the current scale, half the scale long (at most 12 px) and scaled by modulus. The WTMM are the maxima of the modulus along each maxima line (xsmurf's **ssm**).

Anisotropy (Spectrum section) plots the WTMM statistics of Arnéodo, Decoster and Roux (2000, §6): the density $P_a(\mathcal{A})$ at several scales, flat for an isotropic field; the WTMM of one scale in the gradient plane (T_{ψ_1}, T_{ψ_2}), round for an isotropic field; and the density of $\log_2 \mathcal{M}$ in four axial sectors of $\mathcal{A}$ (mod 180° , centred on $0^\circ, 45^\circ, 90^\circ, 135^\circ$), whose shapes coincide when $\mathcal{M}$ and $\mathcal{A}$ are independent.

Frames. Angles are shown in the pixel frame or, for a georeferenced dataset, as azimuths: from grid north for a projected CRS (from the directions its axes run), from true north by adding the meridian convergence at each point, and from true north directly on a longitude/latitude grid. A dataset without a CRS (a sensor swath, a pixel grid) offers the pixel frame only.

4.2 wtmm2d.roi — WTMM over one ROI window

The parent's WTMM re-pointed at a native-resolution window: the knobs of **wtmm2d** plus the window itself. Used by the overview drill-down: draw an ROI on the decimated whole-extent, compute native.

Parameter	Default	Range	Meaning
<code>roi_row, roi_col</code>	0	px	window origin (native pixels)
<code>roi_h, roi_w</code>	512	≥ 8 px	window size
<code>boundary</code>	auto	auto / reflective	boundary handling at the window edge

4.3 mz_edges — Mallat–Zhong dyadic multiscale edges

Mallat and Zhong's multiscale edges ("Characterization of Signals from Multiscale Edges", 1992), computed with the authors' own algorithm: LastWave's **dwtrans2d** package (Bacry, Mallat, Zhong, Hwang et al.), ported to numba and matching its C bit for bit on square fields. The dyadic wavelet transform is computed à trous by direct FIR convolution with the paper's Table I filters ($h = \{\frac{1}{8}, \frac{3}{8}, \frac{3}{8}, \frac{1}{8}\}$ and a two-tap derivative g). The edges at scale 2^l px are **extrema2**'s gradient-modulus maxima: the gradient direction snapped to the row or column axis, a neighbour whose gradient points the opposite way counted lower, and a gap-filling pass at the finest scale; they are linked into per-scale H-lines (**line_id**). Stored moduli carry LastWave's normalization (factors close to $1/\lambda_l$). **Border mirror** runs the whole algorithm on the field mirrored to twice its size, for rasters whose edges do not wrap; **periodic** is LastWave's own mode. Two defects of the C are corrected: non-square fields and a single level ($J = 1$). **Algorithm printed** selects the paper's printed algorithm as the research port implements it

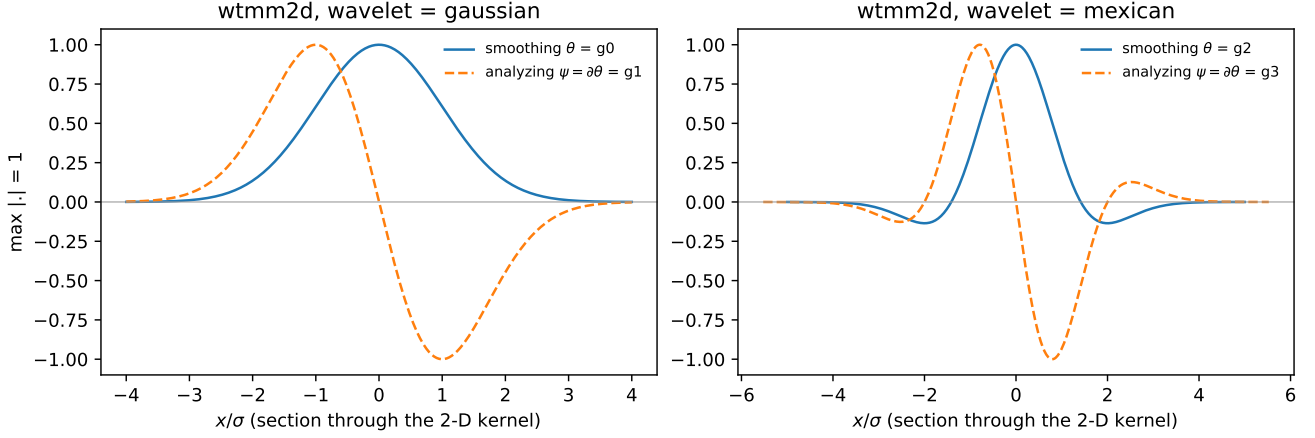


Figure 1: **wtmm2d**: the smoothing function θ and the analyzing wavelet $\psi = \partial\theta$ on the same axes (sections through the 2-D kernels, x in units of the smoothing σ). The wavelet is the derivative of the smoothing function, so the modulus maxima of ψ are the edges of θ -smoothed data.

(Fourier-domain filters on the mirrored torus), kept for comparison. Fields with nodata pixels are refused, and numba is required.

Where the edges are drawn. In the authors’ transform a sample at index j describes position $j - \frac{1}{2}$, so maxima and the coarse channel from the second level up are drawn half a pixel west and north of their index. At the finest level the two gradient components sit half a pixel apart (W_x at $(j - \frac{1}{2}, i)$, W_y at $(j, i - \frac{1}{2})$), as in the authors’ convention; **Co-locate level 1** averages each onto $(j - \frac{1}{2}, i - \frac{1}{2})$ for detection only, and the stored values the reconstruction uses are unchanged.

Reconstruction from the edges. The Outputs rows show *coarse* (the coarse channel S_{2^J} in the data’s units), *thumbnail* (S_{2^J} sampled every 2^J pixels, each block centred on its sample), *recon (edges + coarse)*, *recon (edges only)* (with $S = 0$) and *residual* (the data minus the reconstruction). The reconstruction is LastWave’s **e2recons**: an initial pass, then alternating projections. Each iteration analyses the current image, restores the stored values at the maxima and spreads each correction between neighbouring maxima along rows (W_x) and columns (W_y) as a sinh interpolation with decay $e^{-\kappa/2^l}$ per sample (row and column ends pinned to zero, as in the C), optionally clips the modulus to decrease away from each maximum, and synthesizes with the coarse channel held fixed. $\kappa = 1$ is the paper’s constant; LastWave uses $\kappa = 2 \ln 5.8 \approx 3.516$, which reconstructs markedly better when the coarse channel is pinned and slightly worse for edges only.

The reconstruction knobs live in the right panel’s *Reconstruction* section, shown when **mz_edges** is in the layer’s rack. Showing the recon row computes a one-iteration preview, and Decay κ , Clip and Coarse recompute it as they change. *Run* continues from the preview, either to convergence (the relative change falls under the tolerance, the iteration cap is reached, or the constraint residual rises three iterations in a row, which returns the best iterate) or for a fixed count, with progress and Stop. With **Live** off, no reconstruction row computes on its own: each shows a reconstruction already cached for the current knobs (the recon row also its cached preview), else the data, and reads “press Run” until Run computes the row to show. Each recon row reads its iterations, why the run stopped and its SNR against the data (mean removed; for edges only, against the data minus the synthesised coarse part). ROI results do not reconstruct yet (their rows say so), and the thumbnail shows in the 2-D view. Under *printed* the reconstruction is the research port’s alternating projections with its own **Mode**, run for the iteration count when a row is shown.

Parameter	Default	Range	Meaning
n_levels (Levels J)	4	[1, 10]	dyadic levels; scales $2, 4, \dots, 2^J$ px
algorithm	lastwave	2 choices	the authors’ algorithm, or the paper’s printed algorithm (research port)
border	mirror	mirror / periodic	the algorithm on the mirrored field, or LastWave’s periodic mode

<code>colocate_l1</code> (Co-locate level 1)	off	bool	detect the finest level on co-located gradient components
<code>dither</code> (printed)	off	bool	$\pm \frac{1}{2}$ LSB dither before analysis
<code>interpolate</code> (printed)	off	bool	subpixel refinement of each maximum along its gradient
<code>show</code> (<i>view</i>)	edges	6 outputs	the raster shown in place of the data (the Outputs rows)
<code>recon.live</code> (Live, <i>view</i>)	on	bool	reconstruction rows compute on their own; off: they wait for Run
<code>kappa</code> (Decay κ , <i>view</i>)	1.0	[0.1, 10]	decay of the correction between maxima, $e^{-\kappa/2^l}$ per sample; 1 = the paper, 3.516 = LastWave
<code>clip</code> (<i>view</i>)	off	bool	clip the modulus to decrease away from each maximum (LastWave’s clipping)
<code>run_mode</code> (<i>view</i>)	converge	2 choices	run to convergence, or a fixed number of iterations
<code>iterations</code> (<i>view</i>)	20	[1, 500]	the total count (fixed) or the cap (converge), the preview’s iteration included
<code>tolerance</code> (<i>view</i>)	10^{-3}	$[10^{-6}, 10^{-1}]$	relative change of the image that counts as converged
<code>coarse</code> (<i>view</i>)	full	full / thumbnail	the coarse channel a reconstruction pins: S_{2J} , or the one decoded from its thumbnail
<code>mode</code> (printed, <i>view</i>)	separable	3 modes	how the research port’s projection interpolates between the maxima

4.4 wavelet_skeleton — medial-axis (modulus-minima) extractor

Tang–You 2003 (the constructed compact-support wavelet κ : support radius = scale, maxima-pair separation = scale, width- and gray-invariant) + You et al. 2006 (modulus *minima* sit on the medial axis, scale-independently). A detector, not statistics: directional minima gated by a flanking ridge *pair* (each side within reach $\sim s$ must clear `edge_frac` of the peak modulus), valley contrast (`t_frac` of the flanking ridge — the paper’s threshold T made local), and pair dominance (a pair under `pair_dom` of the strongest reachable ridge is someone else’s slope). Border and nodata neighborhoods (one kernel support) are distrusted. On `input=gradient` the detector runs over $\|\nabla s\|$, where a diffused scarp (an *edge* in the signal) becomes a ribbon whose axis is the fault trace. Output is single-scale extrema (H-lines via `line_id`).

Parameter	Default	Range	Meaning
<code>s1</code> (s_1)	6	[2, 64] px	stage-1 scale = kernel support radius; should cover the ribbon width
<code>s2</code> (s_2)	6	[2, 64] px	refinement-stage scale
<code>t_frac</code> (T/ridge)	0.5	[0.1, 0.9]	keep minima below this fraction of their own flanking ridge
<code>edge_frac</code> (Edge/peak)	0.1	[0.01, 0.6]	flanking-pair significance vs the <i>peak</i> modulus; lower to admit faint features
<code>n_stages</code>	2	[1, 5]	WT passes (1 = detection only; each further pass re-runs on the skeleton image)
<code>input</code> <code>wavelet</code>	signal tang-you	signal / gradient tang-you / gaussian / bspline2	ribbons in the signal, or scarps via $\ \nabla s\ $ the papers’ own comparison set; theorems hold for κ only
<code>pair_dom</code> (Pair dom.)	0.5	[0, 0.9]	pair-dominance gate; 0 disables

4.5 cdf_edges — complex cross-diffusion edges

One complex cross-diffusion evolution (Gilboa small- θ ; Barbeiro 2023 for the scale-space reading), snapshotted at the dyadic schedule $n = \sigma^2/(2 \cos \theta)/dt$: the Re channel evolves as the Gaussian-smoothed image and $\text{Im} / \theta \approx$

$t \cdot \text{LoG}$ (machine-pinned), so one evolution *is* a Mexican-hat scale space. Edge modes: `mz` = grad-Re NMS maxima (*our* graft — the M–Z definition applied to a different generator, making `cdf_edges` directly comparable to `mz_edges` at matched scales); `marr` = Im zero crossings (the CDF authors’ own edge concept — Barbeiro’s v “in the role of edge detector”). The nonlinear variant modulates diffusivity by the Im channel (Perona–Malik-style edge stopping through k). Final Re (`filtered`) and Im (`edge_channel`) rasters ride the result. Scales are effective Gaussian σ in px.

Parameter	Default	Range	Meaning
<code>n_levels</code> (Levels J)	4	[1, 8]	dyadic sigmas $1, 2, \dots, 2^{J-1}$ px; cost is quadratic in the top σ
<code>variant</code>	linear	linear / nonlinear	LCDF, or NCDF with edge-stopping
<code>k_edge</code> (k)	1.0	[0.001, 1000]	Perona–Malik edge threshold, image units (nonlinear only)
<code>theta</code> (θ)	0.1047	[0.001, 0.5] rad	complex-diffusion angle (small- θ regime; $\pi/30$ default)
<code>dt</code>	0.2	[0.001, 0.24]	explicit-Euler step (stability-capped)
<code>edge_mode</code> (Edges)	<code>mz</code>	<code>mz</code> / <code>marr</code>	Canny/M–Z maxima of Re, or Marr zero crossings of Im
<code>detector</code>	<code>nms</code>	<code>nms</code> / <code>follow</code>	maxima detector
<code>show</code> (<i>view</i>)	<code>edges</code>	<code>edges</code> / <code>filtered</code> / <code>edge_channel</code>	the edge stack, the final Re, or the final Im (the Outputs rows; the <i>edges</i> row hides the maxima over either raster)
<code>interpolate</code>	<code>off</code>	<code>bool</code>	sub-pixel refinement of the maxima
<code>floor</code>	0.05	[0, 0.5]	detection floor (fraction of the per-snapshot maximum)

4.6 `pm_edges` — Perona–Malik anisotropic diffusion edges

One Perona–Malik (1990) evolution — real, gradient-driven conduction $g(|\nabla I|)$, $\lambda \leq \frac{1}{4}$, adiabatic borders — snapshotted at the same nominal dyadic σ schedule as `cdf_edges`, with gradient-NMS maxima per snapshot. What it adds: past the flux peak $\phi(s) = s g(s)$ the diffusion runs locally *backward*, sharpening edges while the discrete maximum principle keeps it stable, so edges stay sharp and in place across the stack. Scales are nominal effective σ (the $K \rightarrow \infty$ heat-equation limit); `filtered` (the final diffused raster) rides the result.

Parameter	Default	Range	Meaning
<code>n_levels</code> (Levels J)	4	[1, 8]	dyadic nominal sigmas
<code>k_edge</code> (K)	1.0	[0.001, 1000]	edge threshold in image units: contrasts above K sharpen, below it blur
<code>g</code>	<code>exp</code>	<code>exp</code> / <code>frac</code>	conduction function (the paper’s two)
<code>lam</code> (λ)	0.2	[0.001, 0.25]	explicit step (stability needs $\lambda \leq \frac{1}{4}$)
<code>show</code> (<i>view</i>)	<code>edges</code>	<code>edges</code> / <code>filtered</code>	canvas shows the edge stack, or the diffused raster (the Outputs rows; the <i>edges</i> row hides the maxima over it)
<code>detector</code>	<code>nms</code>	<code>nms</code> / <code>follow</code>	maxima detector
<code>interpolate</code>	<code>off</code>	<code>bool</code>	sub-pixel refinement of the maxima
<code>floor</code>	0.05	[0, 0.5]	detection floor (fraction of the per-snapshot maximum)

5 Decompositions

Standalone transforms heading their own chain, displaying through the ordinary raster path. Every output is computed once and cached; which one is on screen is a *view* choice. To keep an output as data of its own, fork it (§1).

Group (`tucker_HOOI_HOSVD`, `ssa2d`; *view*): which components form the reconstruction — `all` (the whole kept reconstruction, exactly), or a list such as `1-3, 5`. **recon** shows the sum of the group; **residual** shows the data minus that sum (data = recon + residual of the whole kept reconstruction). So removing just the first component is Group 1 with Show = residual. The layer row names it: **recon** [1-3, 5] (88%), **data** — [1] (42%) (the group’s share). An unreadable group shows the whole reconstruction and says so. The Components window picks the group by clicking thumbnails.

5.1 pca — principal components of a multi-band field

Principal component analysis over the bands of a multi-component field ($n_y, n_x, n_c \geq 2$) (covariance-trick PCA); shows one component as a raster.

Parameter	Default	Range	Meaning
n.components (Components)	6	[1, 64]	components computed
component (Show PC; <i>view</i>)	1	[1, 64]	the component shown
standardize	off	bool	scale each band to unit variance first (correlation PCA)

5.2 tucker_HOOI_HOSVD — delay-embedded Tucker decomposition (Tucker-SSA)

A Tucker decomposition of a delay embedding of the field — Broomhead–King singular-spectrum analysis carried to 2-D and multiband data by a Tucker decomposition instead of one SVD. (It was first named `tucker_havok`; HAVOK’s regression on the delay coordinates was never part of it.) Nothing is ever materialized: bases come from Grams of contiguous window slabs, and the reconstruction averages each pixel over the windows covering it.

- **Embedding.** `delay_2d` (default): *both* axes delayed — every $L \times L$ patch ($L = \text{Delays}$), tensor (`delay_y`, `delay_x`, `y'`, `x'` [, band]), so no image axis plays “time”. `delay`: a 1-D Hankel embedding along one axis (Tape axis) — for data where one axis really is time (e.g. a reconstructed tape); on an ordinary 2-D field it biases the result toward that axis. `none`: the array’s own modes (rows, columns[, band]).
- **Method** — a two-state button. **HOSVD**: one pass, each mode’s leading singular vectors. **HOOI**: that HOSVD refined by *HOOI sweeps* alternating passes, each mode re-derived with the others fixed (monotone in the kept core energy; the fast path equals a materialized HOOI to 10^{-8}).
- **Components** (at most 32, largest core-energy share first): under `delay_2d` one per kept (vertical, horizontal) delay-pattern pair (a, b); *Orientation combined* adds each pair to its twin (b, a) ($r(r+1)/2$ components for delay rank r). Tape: one per delay pattern; none: one per rows-mode vector. They sum to the reconstruction.
- Separable by construction: a pattern is a vertical pattern $\otimes$ a horizontal one, so an oblique structure spreads over several pairs. `ssa2d` is the rotation-invariant alternative.

Parameter	Default	Range	Meaning
embed (Embedding)	<code>delay_2d</code>	<code>delay_2d</code> / <code>delay</code> / <code>none</code>	see above
axis (Tape axis)	<code>rows</code>	<code>rows</code> / <code>cols</code>	the time axis of the 1-D tape (greyed out unless <code>delay</code>)
n_delays (Delays)	32	[2, 512]	embedding dimension L (patch $L \times L$ under <code>delay_2d</code>); the lag between delays is 1 px
rank_delay (Rank: delay)	4	[0, 512]	delay patterns kept (both delay modes under <code>delay_2d</code>); 0 = full
rank_time, rank_space	0	[0, 10^5]	ranks of the rows/cols (<code>y'/x'</code>) modes; 0 = full

rank_band (Rank: band)	0	[0, 64]	rank of the band mode (multiband only); 0 = full
method (Method)	HOSVD	HOSVD / HOOI	the algorithm (a toggle button)
sweeps (HOOI sweeps)	2	[1, 16]	HOOI refinement passes (greyed out under HOSVD)
show (<i>view</i>)	recon	recon / residual / component	what the canvas shows
component (<i>view</i>)	1	[1, 64]	the component shown (Show = component)
pairs (Orientation; <i>view</i>)	separate	separate / combined	component numbering under delay_2d; also what a Group refers to
group (Group; <i>view</i>)	all	text	the components the recon sums (above)
cluster (<i>view</i>)	1	[1, 64]	the cluster shown (largest share first)
cluster_distance (<i>view</i>)	0.5	[0.05, 0.95]	where the clustering tree is cut

Superseded: **tucker_havok** stays registered (browser Dev group) so saved projects open and resolve exactly as before; its single *HOOI sweeps* knob chose the algorithm (0 = HOSVD).

5.3 ssa2d — 2D singular-spectrum analysis

Golyandina & Usevich (2010). Every $L_r \times L_c$ window of the image is one column of the Hankel-block-Hankel matrix $\mathbf{W}$ ($L_r L_c \times K_r K_c$, $K = N - L + 1$); *one* SVD $\mathbf{W} = \sum_i \sqrt{\lambda_i} U_i V_i^T$ gives the eigentriples. Reshaped, U_i is an **eigenarray** — an $L_r \times L_c$ pattern of any shape, not restricted to separable products, which makes 2D-SSA invariant to rotations of the image axes — and V_i a **factor array** ($K_r \times K_c$). Each rank-1 term, averaged back onto the image (every pixel over the windows covering it), is an **elementary component**; all of them sum to the image. Ranks (the paper’s §4, pinned by tests): an oblique cosine $\cos 2\pi(\omega_x k + \omega_y l)$ is rank 2, a product of cosines rank 4, a sum of m 2-D exponentials rank m .

Solvers. None forms $\mathbf{W}$.

- *Lanczos* (the default), *PROPACK* and *randomized* use FFT products: $\mathbf{W}^T u$ and $\mathbf{W} v$ are 2-D correlations of the image, done at its own size, whose kept parts never wrap (Korobeynikov 2010; Golyandina et al. 2015). They find only the k leading eigentriples — ARPACK’s Lanczos on the implicit $\mathbf{W}\mathbf{W}^T$; PROPACK’s bidiagonalization of $\mathbf{W}$, with full reorthogonalization (its default partial one returns ghost copies of the leading triplet when the 32-bit FFT products are adjoint only to $\sim 10^{-7}$); or a randomized range finder whose power iterations are re-orthonormalized after every product (Halko et al. 2011; Lopes et al. 2024). Components are FFT convolutions of eigenarray and factor array. They agree with the dense route to float32 tolerance. Measured on a 2048^2 field with a 64×64 window and $k = 16$: Lanczos 6 s, randomized 25 s, PROPACK 42 s.
- *Dense*: the lag covariance $\mathbf{W}\mathbf{W}^T$ ($(L_r L_c)^2$) accumulated from blocks of window rows, its full eigenbasis, then $\mathbf{W}^T U$ in the same blocks; float64 BLAS, equal to an explicit SVD of $\mathbf{W}$ to 10^{-9} . Limited to windows of 1600 entries (40×40 , about 4 s on a 256^2 field). An FFT solver hands over to it when the kept count reaches the window size.

The shares $\lambda_i / \text{tr}(\mathbf{W}\mathbf{W}^T)$ are exact on every route (the trace is the sum of the squared pixels weighted by their window counts). Each eigenarray is signed so that its largest entry is positive. Large outputs are stored at the app’s precision. Single-band fields only. NaN pixels are filled with the finite mean for the decomposition and re-masked in every output. Also returned: the **w-correlation** matrix (the paper’s Def. 3.18) of the kept components — the grouping aid shown in the Components window.

Keeping and grouping (Lopes et al. 2024). *Keep: energy* treats **n_components** as the most to find and keeps the fewest whose cumulative share reaches the threshold (the row note says when it was not reached). This 2D-SSA is uncentred, so on a raw field the first eigentriple, the mean, often carries most of the energy on its own. *Show: cluster* shows one group of an automatic grouping: average-linkage clustering on $1 - |\text{w-correlation}|$, cut

at the cluster distance, clusters ordered by share. The cluster knobs are view settings, so changing them never re-decomposes.

Edges. A pixel near the edge of the analysed region is covered by fewer windows. On a region of interest the tool reads a margin of real data around it (`margin_windows` windows of $\max(L_r, L_c) - 1$ pixels; at one window every ROI pixel is fully covered) and crops back. On the whole field there is no data beyond the edge. Lopes et al.’s mirror extension is available (*Whole-field edges: mirror*), but off by default: in 2-D a reflected wave is a different wave (an oblique wave flips a frequency component), so the extension adds rank. On two test waves it raised the edge error 3–18 $\times$, where the real-data margin of an ROI lowered it by about 10%.

Parameter	Default	Range	Meaning
<code>rows_window, cols_window</code>	16	[2, 512]	window $L_r \times L_c$ ($L_r L_c \leq 1600$ for dense)
<code>n_components</code> (Components kept)	16	[1, 64]	leading eigentriples kept (the most, with Keep: energy)
<code>keep</code>	count	count / energy	keep <code>n_components</code> , or the fewest reaching the energy threshold
<code>energy_threshold</code> (Energy)	0.9	[0.5, 0.9999]	cumulative share to reach (Keep: energy only)
<code>solver</code>	lanczos	lanczos / propack / randomized / dense	the eigentriple route (above)
<code>power_iterations</code>	2	[0, 10]	randomized solver only
<code>margin_windows</code> (Edge margin)	1	[0, 4]	ROI margin in windows; also the mirror width
<code>edge_extension</code> (Whole-field edges)	none	none / mirror	mirror-extend the whole field (off: it adds rank in 2-D)
<code>show</code> (<i>view</i>)	recon	recon / residual / component / cluster	what the canvas shows
<code>component</code> (<i>view</i>)	1	[1, 64]	the elementary component shown
<code>group</code> (Group; <i>view</i>)	all	text	the components the recon sums (above)
<code>cluster</code> (<i>view</i>)	1	[1, 64]	the cluster shown (largest share first)
<code>cluster_distance</code> (<i>view</i>)	0.5	[0.05, 0.95]	where the clustering tree is cut

6 Microcanonical tools

Per-pixel singularity exponents $h(x)$ from the model $T(x, r) \approx \alpha(x) r^{h(x)}$ (Turiel 2008), $D(h)$ by direct histogram — the formalism that is *not* canonical/Legendre. One tool per functional; both expose the Pont 2006 estimator pair: **regression** (per-pixel log–log slope over the scale grid; the prefactor cancels exactly; resolves both limbs of $D(h)$) and **punctual** (the finest scale alone, ensemble-mean normalized; sharpest localization, the most-singular-set extractor; cannot resolve the right limb; carries an $\alpha(x)/\langle\alpha\rangle$ bias). Both estimators share the same cross-scale no-support mask — they agree on *where* an exponent exists and differ in *how*. Under punctual, $\kappa/\mathbf{n_scales}$ shape only the mask and `r2_map` is honestly all-NaN.

6.1 holder_measure — gradient-measure route

T = positive unit-mass kernel over $\|\nabla s\|$ (finite differences first, then g_0 -class smoothing). For measure-like / edge-sparse fields; slope = h . The scale knob r_1 is the kernel’s *dilation scale* (for the Gaussian: its standard deviation) — positive kernels have no zero crossings, and their resolution floor is the pixel, except for the compactly supported q-Gaussian (below). Kernel trade-off (Turiel App. A): Gaussian resolves every h ; Lorentzian truncates at $h < 2\beta - d$ but localizes sharpest; q-Gaussian interpolates; `frac_gaussian` fractionalizes the envelope, $e^{-\rho^n/2}$ ($n=2$ is the Gaussian exactly).

The q-Gaussian is Tsallis's $e_q^{-\rho^2/2r^2}$ with $e_q^x = [1 + (1 - q)x]_+^{1/(1-q)}$ (Borges et al. 2004, eq. 9), for $-1 \leq q \leq 3$. Above $q = 1$ it has power-law tails (Student-t / Lorentzian: $q \equiv 1 + 1/\beta$ with ρ^2/r^2 rescaled); $q = 1$ is the Gaussian; below $q = 1$ it is *compactly supported*, radius $r\sqrt{2/(1-q)}$ — $q = 0$ is the Epanechnikov paraboloid, $q = -1$ a half-dome of radius r , and $q \rightarrow -\infty$ the Dirac δ . For $q \geq 2$ the 2-D kernel has no finite mass (the Lorentzian $\beta \leq 1$ case), so the result depends on the field size; the q knob shows a warning there. The compact kernel (below $q = 1$) is the steepest decrease there is, so it carries a resolution floor: the r_1 knob warns when its support disc, of radius $r/\sqrt{\beta(1-q)}$ for width β , is less than 2 px across, i.e. below $r_1 = \sqrt{\beta(1-q)}$ (at most 1 px at the default width).

6.2 holder_multiaffine — wavelet-projection route

$T = |\psi_r * s|$ with a zero-mean wavelet applied to the *raw signal* (derivative order lives inside the kernel). For smooth / function-class fields (DEMs); slope = γ , with $h_{\text{measure}} = \gamma - 1$. Here r_1 is the *zero-crossing radius*: every wavelet in the menu is calibrated to change sign at radius r , so for a wavelet with one crossing the fig-2 minimum-resolution rule reads $r_1 \geq 1$ (more crossings, or a compact support, raise it: *Resolution floor* below). The g-ladder is the vanishing-moment axis (order n measures only $\gamma < n$): **g1/g3** are the fractional family at $n=1,3$; **g2** is the Ricker; **lorentzian_marr** is the Laplacian (Marr operator) of the Lorentzian envelope (true tail $\rho^{-2\beta-2}$, truncation $\gamma \geq 2\beta$); **frac-gaussian** makes the order a real knob ($\psi \propto {}_1F_1(\frac{n+2}{2}; 1; -\rho^2/2\sigma^2)$, $n=2$ reproduces the Ricker exactly): n is a fractional *differentiation* order ($\hat{\psi} \sim |k|^n$ at low frequency), so exact moments vanish only for the integer orders below n — a fractional n is not “ n vanishing moments” — and $\gamma < n$ is measurable.

q_mexican is the q-Mexican hat of Borges et al. (2004, eq. 15): the second derivative of the q-Gaussian *raised to the power 2 - q*. In 2-D (the Laplacian) this gives

$$\psi_q(\rho) \propto [1 - (2 - q)b\rho^2] [e_q^{-b\rho^2}]^q, \quad b = \frac{1}{(2 - q)r^2},$$

so the zero crossing sits at $\rho = r$ for every q , and $q = 1$ is the Ricker exactly. The paper's $q < 3$ is the 1-D bound: in 2-D the Laplacian of $[e_q]^{2-q}$ vanishes identically at $q = 2$ and never changes sign above it, so the knob runs $-1 \leq q \leq 1.95$. For $q \geq 0$ the wavelet is bounded; for $q < 0$ it diverges at its cutoff (still integrable) and the knob shows a warning. Because $[e_q]^{2-q}$ is itself a q' -Gaussian with $q' = 1/(2 - q)$, this is the family of Laplacians of q-Gaussians under a relabelled index: **q_mexican** at q equals **lorentzian_marr** at $\beta = (2 - q)/(q - 1)$ for $1 < q < 2$ (e.g. $q = 1.5 \equiv \beta = 1$), with tail $\rho^{-2/(q-1)}$ and truncation $\gamma \geq 2(2 - q)/(q - 1)$.

The q width β (both q-kernels) is the β of $e_q^{-\beta x^2}$, a toggle away from the paper's two choices.

- **Fixed** (default $\beta = \frac{1}{2}$): the unit-variance Gaussian per component at $q = 1$, in any dimension. The scale r keeps its meaning at $q = 1$: $\sigma = r$ for the q-Gaussian, the zero crossing at r for the q-Mexican hat. At other q the q-Mexican hat's zero crossing drifts to $r/\sqrt{2\beta(2 - q)}$.
- **q-paired**: Borges et al.'s $\beta = 1/(3 - q)$ holds the q-Gaussian's *escort* (q-)variance $\int x^2 [e_q]^q / \int [e_q]^q$ at one. In d dimensions that variance is $1/(\beta(d + 2 - dq))$ per component, so the pairing is $\beta = 1/(d + 2 - dq)$: $1/(3 - q)$ in 1-D and $\beta(q) = 1/(2(2 - q))$ here.
- The q-Mexican hat's zero crossing sits exactly at that escort RMS radius, so the pairing also pins it (at r , for every q : the tool's classic calibration). The two settings agree at $q = 1$.
- The pairing exists exactly where the 2-D q-Mexican hat is admissible, $q < 2$: the escort variance and the zero mean both need $d + 2 - dq > 0$. A q-paired q-Gaussian therefore refuses $q \geq 2$.
- In real space the normalisation is done numerically at every scale, so the paper's closed-form constants are not used. (In 2-D they happen to be elementary: $A_q = \sqrt{\beta(3 - q)(1 + q)}/\pi$ for the q-Mexican hat, $B_q = \sqrt{\beta(3 - q)}/\pi$ for the modulated q-Gaussian. The flat Jacobian $\rho d\rho = du/2\beta$ turns every integral into a Beta function with integer arguments, where 1-D keeps $\Gamma(\frac{1}{2})$ -type ratios.)

Resolution floor on r_1 . A discretized wavelet must separate its positive and negative parts, so its zero crossings set the minimum attainable resolution, and more crossings cost resolution (Turiel et al. 2008, fig. 2 and §4.2.1); Turiel (2009) puts a Mexican hat's inner minimum scale at “several pixels”. Neither gives a number. The tools

read the rule as: every sign lobe at least 2px across at r_1 , the central disc counted by its diameter. The edge of a compact support is not a sign change, but it closes the last lobe. The r_1 knob warns below the floor and never refuses.

- One crossing at r (**g1**, **g2**, **lorentzian_marr**, **frac_gaussian** for $n \leq 2$): the floor is $r_1 = 1$, as before.
- More crossings: **g3** 1.32px (its ring is $1.52r$ wide); **frac_gaussian** at $n = 4$ and 5, 1.41 and 1.46px.
- **q_mexican**: its crossing sits at $\rho_0 = r/\sqrt{2\beta(2-q)}$ (r when q-paired) and, below $q = 1$, its support ends at $\rho_0\sqrt{(2-q)/(1-q)}$. The negative ring between them narrows as q falls, and below $q \approx 0.875$ (q-paired) it is the narrowest lobe: floor 2.73px at $q = 0.5$ and 4.42px at $q = 0.1$. A fixed width β is a pure dilation of the wavelet, so it rescales the floor by $\sqrt{2\beta(2-q)}$.
- **holder_measure**: only the compact q-Gaussian has a floor (above).

Wavelet space: real or Fourier (both tools). *Real* (the default, Turiel’s construction) samples each kernel on the pixel grid and normalizes it there. *Fourier* evaluates the same continuous kernel’s closed-form 2-D transform at $r\mathbf{k}$: a Matérn function of k for a q-Gaussian with $q > 1$ (and the Lorentzian, $q = 1 + 1/\beta$), a Jahnke–Emde function for $q < 1$, $k^n e^{-k^2/4u_0}$ for the fractional family, evaluated stably at every q . The kernel is then band-limited rather than sampled.

- Measure route: unit mass exactly. Multiaffine route: zero mean exactly, and the kernel is the L^1 -dilated continuous wavelet, so T equals the real construction’s times the mother wavelet’s L^1 norm (a constant that cancels in every slope) wherever both are exact.
- They differ where sampling matters: at the smallest scales, for compact kernels (the Fourier one rings slightly), and for heavy tails, which real space cuts at the edge of the padded grid (about 10^{-4} at the default padding for a $\rho^{-6.7}$ tail).
- Fourier is refused on the measure route for **frac_gaussian** (no closed-form transform), and for a q-Gaussian at $q \geq 2$ or a Lorentzian with $\beta \leq 1$ (no finite mass to normalize by).

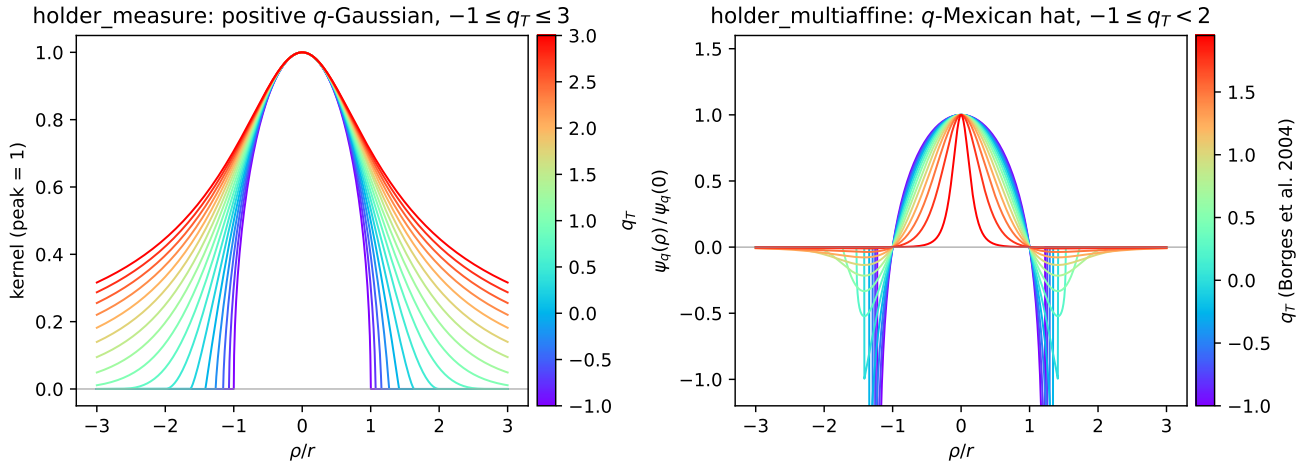


Figure 2: The q family. Left: **holder_measure**’s positive q -Gaussian, $-1 \leq q \leq 3$ (compact below $q = 1$). Right: **holder_multiaffine**’s q -Mexican hat (Borges et al. 2004, in 2-D), $-1 \leq q < 2$; every curve crosses zero at $\rho = r$, and for $q < 0$ the negative lobe diverges at the cutoff. Profiles scaled to 1 at the centre.

The same r_1 value is not numerically comparable across the two tools, and γ values are not comparable across wavelets.

Parameter	Default	Range	Meaning (both holder tools)
estimator	regression	regression / punctual	Pont estimator pair (see above)

<code>r_min</code> (r_1)	1.0	[0.1, 32] px	finest scale (dilation scale / zero-crossing radius per tool); warns below the resolution floor
<code>kappa</code> ($\kappa = r_2/r_1$)	8	[1.1, 200]	scale-range width; the knob that changes the physics
<code>n_scales</code>	6	[2, 64]	samples in the range (nearly irrelevant vs κ ; keep small)
<code>wavelet</code>	(per tool)	method-true menu	see tool text
<code>q_tsallis</code> (Tsallis q , q_T)	1.5	$[-1, 3]$ / $[-1, 1.95]$	q-family shape, measure / multiaffine (q-gaussian / q-mexican only); warns at measure $q \geq 2$, multiaffine $q < 0$
<code>beta</code> (β)	1.0	[0.1, 8]	Lorentzian tail exponent (Lorentzian family only)
<code>frac_n</code> (n)	2.0	[0.05, 8]	fractional order (frac-gaussian only)
<code>q_pairing</code> (q width)	fixed	fixed / q-paired	the q-kernel's width β : fixed, or $\beta(q) = 1/(2(2 - q))$ (below); greyed out unless q-gaussian / q-mexican
<code>q_beta</code> (q β)	0.5	[0.05, 5]	the fixed width of $e_q^{-\beta \rho^2/r^2}$ (greyed out while q-paired)
<code>construction</code> (Wavelet space)	real	real / fourier	sample each kernel on the grid, or build it from its closed-form transform (above)

6.3 `band_recon_measure` / `band_recon_multiaffine` — reconstruction from an h band

The same $h(x)$ engine as the matching holder tool (its params verbatim, shared by identity) + the singularity band $h \in [h_{lo}, h_{hi}]$ and an island sieve, then gradient-restricted Fourier reconstruction (Turiel eq. 65, symmetric-extension border fix). Sweeping the band and watching the reconstruction *is* the workflow; the Band dialog commits the variant matching the layer's producer.

Parameter	Default	Range	Meaning (band knobs; engine knobs above)
<code>h_lo</code> / <code>h_hi</code>	-2 / 0	$[-6, 6]$	the band $h_{lo} \leq h < h_{hi}$, in the estimator's own frame
<code>min_island</code> / <code>max_island</code>	0 / 0	px	drop mask components outside this size range before inversion
<code>connectivity</code>	8	8 / 4	island connectivity (8 keeps diagonal ribbons whole)

Superseded: `holder_map` and `band_recon` (the pre-split conflated tools, estimator $\times$ wavelet crossed) remain registered for saved projects and live in the browser's Dev group; their physics is identical to the split tools at the old defaults.

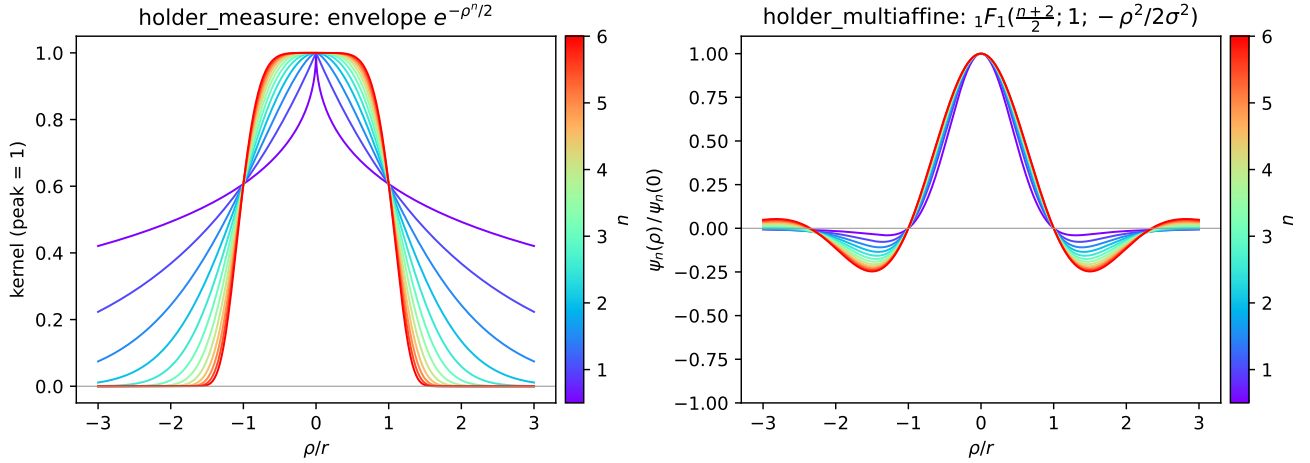


Figure 3: The fractional Gaussians. Left: `holder_measure`'s envelope $e^{-\rho^n/2}$ ($n < 2$ fattens the tail, $n > 2$ flattens the top; every curve passes $e^{-1/2}$ at $\rho = r$). Right: `holder_multiaffine`'s order- n wavelet ${}_1F_1(\frac{n+2}{2}; 1; -\rho^2/2\sigma^2)$, σ set per order so the first zero crossing sits at $\rho = r$.

7 Topology

7.1 `chain_topology` (Transform)

Builds the incidence graph over the WTMM maxima-line skeleton (no parameters). Must follow `wtmm2d`; downstream topology filters query the graph it builds.

7.2 `min_vchains` (Filter)

Keeps H-line points that at least `min_vchains` maxima lines (V-chains) thread. Default UI opens at 1 (show everything computed; turn up to prune).

8 Extrema filters

Act on the per-scale extrema layers of any producing analyzer (`wtmm2d`, `mz_edges`, `wavelet_skeleton`, `cdf_edges`).

Device.param	Default	Range	Meaning
<code>scale_select.scale_idx</code>	0	[0, 255]	pick ONE scale from the stack — <i>the scale scrubber</i> ; a list index, instant re-scale
<code>orientation_wedge.centre</code>	0	deg	keep extrema whose gradient strike falls in wedge
<code>orientation_wedge.half_width</code>	90	[0, 90] deg	wedge half-width (90 = no-op)
<code>orientation_wedge.north</code>	grid	grid/true/magnetic	the NorthRef of <code>centre</code> ; frame errors, multi-degree errors
<code>modulus_threshold.frac</code>	0	[0, 1]	per-point: keep $ W \geq \text{frac} \times \text{scale peak}$
<code>hline_length.min_len/max_len</code>	1 / 0	points	per-line point count range; kills orphan confetti and stitching seams; -1 orphan
<code>hline_modulus.frac</code>	0	[0, 1]	per-line modulus keep-fraction
<code>hline_modulus.mode</code>	sup	sup / sup_all_scales / mean / segments	line statistic: peak at this scale; peak at all scales; mean; mean of segments; threading chains (needs chains); mean per-segment between modulus minima

9 Chain filters (WTMM maxima lines)

Act on cross-scale chains — `wtmm2d` results only (the other analyzers stamp `chains:` `[]`; their cross-scale linking is reserved work).

Device.param	Default	Range	Meaning
<code>chain_holder.cutoff</code>	−3	[−3, 3]	keep chains with Hölder $\geq$ cutoff
<code>chain_holder.estimator</code>	ols	ols / max	per-chain h : log-log OLS slope, or the max-modulus scaling read
<code>chain_modulus.threshold</code>	−99	[−99, 32]	keep chains whose max $\log_2 W \geq$ threshold
<code>chain_length.min_len</code>	2	[2, 64]	minimum scales spanned

9.1 chain.classify

Classifies chains by their Hölder exponent and geometry into step-like ($h \approx 0$: fault-scarp class) vs delta-like ($h \approx -1$: thin-ridge class) lineaments, with a straightness gate; flagged or excluded per `action`, excluded chains optionally drawn as ghosts.

Parameter	Default	Range	Meaning
<code>estimator</code>	ols	ols / max	per-chain h estimator
<code>fit_a_min</code>	2.0	[0.01, 64]	fit floor: scales below this are excluded from the h fit
<code>h_step_lo/hi</code>	−0.15 / 0.15	[−3, 3]	the step-class h window
<code>h_delta_lo/hi</code>	−1.2 / −0.8	[−3, 3]	the delta-class h window
<code>straightness_min</code>	0.85	[0, 1]	end-to-end / arc-length straightness gate
<code>action</code>	flag	flag / exclude	tag matches, or drop non-matches
<code>show_ghosts</code>	on	bool	draw excluded chains desaturated

10 Groups, points, dev

`group.paint` / `group.filter`: the arrangement view’s commit transaction — `group.paint` stamps committed group tags onto chain copies (`spec_json/signature` are written by the commit flow, not hand-edited); `group.filter` keeps or drops one named group.

`backproject`: registers a point catalogue onto a raster layer’s grid (the point-layer transform). `target` names the raster layer; the `_target_*` params are the resolved grid geometry, written by the shell — not hand-edited.

`derived_vectors`: a derivative dataset’s frozen extrema and maxima lines, read from its own file (§1). Placed by the shell (browser Dev group); its one parameter (the file) is hidden.

`stub.wavelet` / `stub_holder` / `stub.wedge`: parameter-schema test fixtures (the browser’s Dev group); no analysis role.

11 Planned: q-generalized wavelets

Not implemented yet — recorded here so the design stays in one place. Borges et al. (2004) generalize the Mexican hat (above), the Morlet wavelet (a q -Gaussian-modulated plane wave, their eq. 23) and the trigonometric functions (an even/odd pair of q -trigonometric wavelets, their §4), and note that “other wavelets, as well as scaling functions, may be generalized along the lines here introduced, for instance, the Meyer wavelet or the sinc function.”

- **q-generalized g1 and g3 for wtmm2d.** WTMM’s analyzing wavelets must stay *derivatives of a smoothing function* ($\psi = \partial\theta$), so that maxima mark edges. The filters are built in the Fourier domain, so they need the 2-D transform of the q -Gaussian, a Hankel transform with Bessel order $1/(q-1) - d/2$, one half lower

than the paper’s 1-D order. That transform is now built and evaluated stably at every q (the Matérn and Jahnke-Emde functions used by *Wavelet space: Fourier*, §6).

- **q-g1**: $\partial\theta_q$ with θ_q the unit-mass q -Gaussian, so the L^1 normalization is unchanged ($\hat{\theta}_q(0) = 1$); it exists for $q < 2$.
- **q-g3**: the power- $(2-q)$ recipe makes the Laplacian’s envelope a q' -Gaussian with $q' = 1/(2-q)$, whose mass is finite only for $q < 3/2$. For $3/2 \leq q < 2$ the wavelet is still finite (k^2 cancels the divergence at $k = 0$), but $\hat{\theta}(0) = 1$ can no longer fix its scale, so it needs another normalization there, such as unit L^1 norm.
- **q-Morlet** and the **q-trigonometric** pair (oriented/directional analysis).
- **q-Meyer** and **q-sinc**, as the paper suggests.

References

- A. Arnéodo, N. Decoster and S. G. Roux (2000), A wavelet-based method for multifractal image analysis. I. Methodology and test applications on isotropic and anisotropic random rough surfaces, *Eur. Phys. J. B* 15, 567–600.
- E. P. Borges, C. Tsallis, J. G. V. Miranda and R. F. S. Andrade (2004), Mother wavelet functions generalized through q -exponentials, *J. Phys. A: Math. Gen.* 37, 9125–9137. doi:10.1088/0305-4470/37/39/006
- A. Turiel, H. Yahia and C. J. Pérez-Vicente (2008), Microcanonical multifractal formalism — a geometrical approach to multifractal systems: Part I. Singularity analysis, *J. Phys. A: Math. Theor.* 41, 015501.
- A. Turiel (2009), Relevance of multifractal textures in static images, book chapter.
- M. Unser and T. Blu (1999), Construction of fractional spline wavelet bases, *Proc. SPIE* 3813 (Wavelet Applications in Signal and Image Processing VII), 422–431.
- M. Unser and T. Blu (2000), Fractional splines and wavelets, *SIAM Review* 42, 43–67.
- S. Mallat and S. Zhong (1992), Characterization of signals from multiscale edges, *IEEE Trans. PAMI* 14, 710–732.
- P. Perona and J. Malik (1990), Scale-space and edge detection using anisotropic diffusion, *IEEE Trans. PAMI* 12, 629–639.
- N. Golyandina and K. Usevich (2010), 2D-extension of singular spectrum analysis: algorithm and elements of theory, in *Matrix Methods: Theory, Algorithms and Applications* (V. Olshevsky, E. Tyrtyshnikov, eds.), World Scientific, 449–473.
- A. Korobeynikov (2010), Computation- and space-efficient implementation of SSA, *Statistics and Its Interface* 3, 257–268.
- N. Golyandina, A. Korobeynikov, A. Shlemov and K. Usevich (2015), Multivariate and 2D extensions of singular spectrum analysis with the Rssa package, *J. Stat. Software* 67(2), 1–78.
- N. Halko, P.-G. Martinsson and J. A. Tropp (2011), Finding structure with randomness: probabilistic algorithms for constructing approximate matrix decompositions, *SIAM Review* 53, 217–288.
- F. Lopes, D. Gibert, V. Courtillot, J.-L. Le Mouél and J.-B. Boulé (2024), On the optimization of singular spectrum analyses: a pragmatic approach, preprint.
- D. S. Broomhead and G. P. King (1986), Extracting qualitative dynamics from experimental data, *Physica D* 20, 217–236.
- L. De Lathauwer, B. De Moor and J. Vandewalle (2000), A multilinear singular value decomposition, *SIAM J. Matrix Anal. Appl.* 21, 1253–1278; and On the best rank-1 and rank- $(R_1, \dots, R_N)$ approximation of higher-order tensors, *ibid.*, 1324–1342 (HOSVD and HOOI).

Appendix: result contracts

- **Extrema schema** (all edge/skeleton analyzers): per-scale `{x, y, mod, arg, line_id}`; `line_id` groups H-lines, `-1` marks isolated points. `scales` is the matching scale array (px).
- **h-map contract** (holder tools): `h_map` displays in place of the field through the ordinary raster path; `r2_map` rides along (all-NaN under punctual). `raster_out` (band tools) displays the reconstruction; `h_map/band_mask` ride for the dialogs.
- **cdf extras**: `filtered` (final Re) and `edge_channel` (final Im) rasters.
- The vector (3-D) view draws the *finest* extrema layer only — put `scale_select` in the chain to choose which.
- **Decomposition extras**: `ssa2d` — `ssa_recon`, `ssa_residual`, `ssa_components` (n, n_y, n_x), `ssa_eigen_share`, `ssa_eigenarrays` (n, L_r, L_c), `ssa_factor_arrays` (n, K_r, K_c), `ssa_w_correlation` (n, n). `tucker_HOOI_HOSVD` — `tucker_recon`, `tucker_residual`, `tucker_components` / `tucker_combined_components` with their energies, core, factors, energy per mode; on a multiband field also `tucker_recon_bands` / `tucker_residual_bands` (n_y, n_x, n_c) (the display shows band 1). A view step writes `raster_out` and, for a named view, `_view_note` (what the layer row appends).
- **Derivative file**: the RasterField npz keys (`schema`, `name`, `values`, `x_axis`, `y_axis`, `units`, `frame_meta`, `provenance`) plus, when it carries vectors, `vec_e_*` (extrema, CSR-packed), `vec_c_*` (maxima lines) and `vec_scales`. A vector-only derivative's raster is all-NaN on the same grid.